

Федеральное государственное автономное образовательное учреждение высшего
образования



**«Московский государственный технический университет
имени Н.Э. Баумана»**

(национальный исследовательский университет)

(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ ИНФОРМАТИКА И СИСТЕМЫ УПРАВЛЕНИЯ
КАФЕДРА КОМПЬЮТЕРНЫЕ СИСТЕМЫ И СЕТИ (ИУ6)

О т ч е т

по лабораторной работе № 1

**Название лабораторной работы: Программирование с использованием
библиотеки QT.**

Дисциплина: Алгоритмизация и программирование

Студент гр. **ИУ6-23Б** _____
(Подпись, дата)

Соболев С.М.
(И.О. Фамилия)

Преподаватель _____
(Подпись, дата)

О.А. Веселовская
(И.О. Фамилия)

Москва, 2025

Часть 1. Создание полиморфного контейнера и исключений

Цель работы: Освоить принципы объектно-ориентированного программирования на языке C++, в частности полиморфизм, наследование и механизм обработки исключительных ситуаций. Разработать контейнер типа очередь с графическим интерфейсом для безопасного хранения и обработки гетерогенных данных.

Задание: Разработать контейнер в виде совокупности классов.

Предусмотреть не менее 2-х исключений. Разработать тестирующую программу с графическим интерфейсом. В отчете представить диаграмму классов приложения и обосновать выбранную структуру представления данных.

Моделировать очередь, в качестве элементов которой могут использоваться числа и строки. Операции: добавление элемента, удаление элемента, печать элементов очереди. Создать класс - потомок, который содержит процедуру сортировки строк по алфавиту. Тестировать полученную модель.

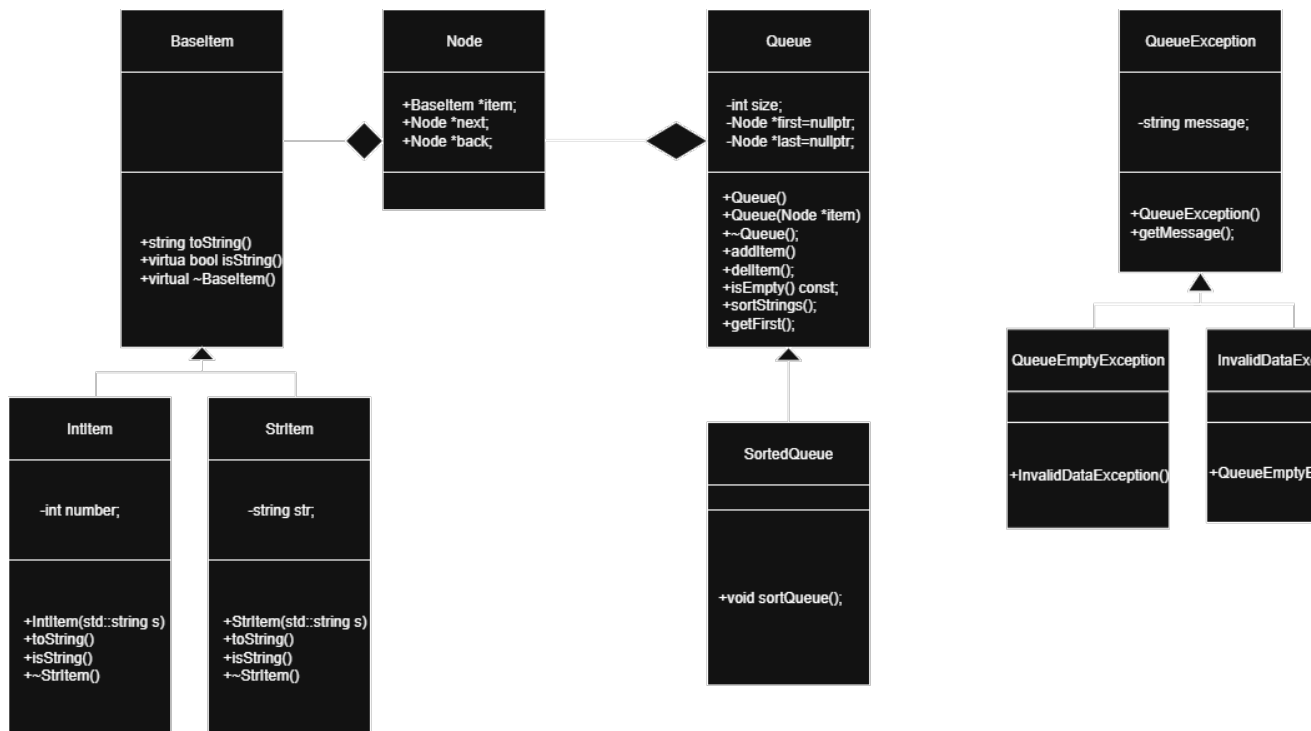


Рисунок 1 – Диаграмма классов

```

1  #ifndef BASEITEM_H
2  #define BASEITEM_H
3
4  #include <string>
5  #include <cstdio>
6  class BaseItem
7  {
8  public:
9      virtual std::string toString() const = 0;
10     virtual bool isString() const = 0;
11     virtual ~BaseItem() = default;
12 };
13
14 class IntItem: public BaseItem
15 {
16     int number;
17 public:
18     IntItem(int n): number(n) {};
19     std::string toString() const override;
20     bool isString() const override;
21     ~IntItem() override;
22 };
23
24 class StrItem: public BaseItem
25 {
26     std::string str;
27 public:
28     StrItem(std::string s): str(s) {}
29     std::string toString() const override;
30     bool isString() const override;
31     ~StrItem() override;
32 };
33 #endif // BASEITEM_H
34
  
```

Рисунок 2 – Файл BaseItem.h

```

1  #ifndef MAINWINDOW_H
2  #define MAINWINDOW_H
3
4  #include <QMainWindow>
5  #include "queue.h"
6  #include <string>
7  #include <QMessageBox>
8  #include "queueexception.h"
9
10  QT_BEGIN_NAMESPACE
11  namespace Ui {
12  class MainWindow;
13  }
14  QT_END_NAMESPACE
15
16  class MainWindow : public QMainWindow
17  {
18  |   Q_OBJECT
19
20  public:
21  |   MainWindow(QWidget *parent = nullptr);
22  |   void updateList();
23  |   ~MainWindow();
24
25  private slots:
26  |   void on_addBtn_clicked();
27
28  |   void on_comboBox_currentIndexChanged(int index);
29
30  |   void on_delBtn_clicked();
31
32  |   void on_sortBtn_clicked();
33
34  private:
35  |   Ui::MainWindow *ui;
36  |   Queue *numbers;
37  |   SortedQueue *strings;
38  |   int indexCB = 0;
39  };
40  #endif // MAINWINDOW_H
41

```

Рисунок 3 – Файл MainWindow.h

```

1  #ifndef QUEUE_H
2  #define QUEUE_H
3
4  #include <string.h>
5  #include <stdio.h>
6  #include "baseitem.h"
7  #include "queueexception.h"
8
9  struct Node{
10     BaseItem *item;
11     Node *next;
12     Node *back;
13 };
14
15
16 class Queue{
17     int size;
18     Node *first=NULLPTR;
19     Node *last=NULLPTR;
20
21 public:
22     Queue(): size(0), first(NULLPTR) {}
23     Queue(Node *item): size(1), first(item) {}
24     ~Queue();
25     void addItem(BaseItem *it);
26     void delItem();
27     bool isEmpty() const;
28     void sortStrings();
29     Node* getFirst();
30 };
31
32
33 class SortedQueue: public Queue{
34 public:
35     void sortQueue();
36 };
37
38

```

Рисунок 4 - Файл Queue.h

```

1  #ifndef QUEUEEXCEPTION_H
2  #define QUEUEEXCEPTION_H
3
4  #include <string>
5
6  class QueueException
7  {
8  |   std::string message;
9  public:
10 |   QueueException(std::string s): message(s) {}
11 |   std::string getMessage();
12 };
13
14 class QueueEmptyException: public QueueException{
15 public:
16 |   QueueEmptyException(): QueueException("Ошибка: попытка удаления из пустой очереди!"){ }
17 };
18
19 class InvalidDataException: public QueueException
20 {
21 public:
22 |   InvalidDataException():QueueException("Ошибка: введено некорректное число!"){ }
23 };
24
25
26 #endif // QUEUEEXCEPTION_H
27

```

Рисунок 5 - Файл QueueException.h

```

1  #include "baseitem.h"
2
3  std::string IntItem::toString() const {
4      return std::to_string(number);
5  }
6  bool IntItem::isString() const {
7      return 0;
8  }
9  IntItem::~IntItem() {
10
11 }
12
13 std::string StrItem::toString() const {
14     return str;
15 }
16 bool StrItem::isString() const {
17     return 1;
18 }
19 StrItem::~StrItem() {
20
21 }
22

```

Рисунок 6 - Файл BaseItem.cpp

```

void MainWindow::on_addBtn_clicked()
{
    QString text = ui->lineEdit->text();
    bool ok;
    int value = text.toInt(&ok);
    if (ui->comboBox->currentIndex() == 0){
        try{
            if (ok) numbers->addItem(new IntItem(value));
            else throw InvalidDataException();
        }
        catch (InvalidDataException& e){
            QMessageBox::warning(this, "Warning", QString::fromStdString(e.getMessage()));
        }
    }
    else{
        strings->addItem(new StrItem(text.toStdString()));
    }
    ui->lineEdit->clear();
    updateList();
}

void MainWindow::updateList(){
    ui->listWidget->clear();
    if (indexCB == 0){
        Node *first = numbers->getFirst();
        Node *current = first;
        while (current != nullptr){
            ui->listWidget->addItem(QString::fromStdString(current->item->toString()));
            current = current->next;
        }
    }
    else{
        Node *first = strings->getFirst();
        Node *current = first;
        while (current != nullptr){
            ui->listWidget->addItem(QString::fromStdString(current->item->toString()));
            current = current->next;
        }
    }
}

```

Рисунок 7 - Файл MainWindow.cpp

```

void Queue::addItem(BaseItem *it){
    if (first == nullptr)
    {
        first = new Node[it, nullptr, nullptr];
        last = first;
        return;
    }
    Node *current = first;
    while (current->next != nullptr) {
        current = current->next;
    }
    current->next = new Node[it, nullptr, current];
    last = current->next;
    size += 1;
}

bool Queue::isEmpty() const{
    if (first == nullptr) return 1;
    return 0;
}

void Queue::delItem(){
    if (isEmpty()) throw QueueEmptyException();
    Node *temp = first->next;
    delete first;
    first = temp;
}

Node* Queue::getFirst(){
    return first;
}

Queue::~Queue(){
    Node *current = first;
    while (current->next != nullptr) {
        Node *temp = current;
        current = current->next;
        delete temp;
    }
}

void SortedQueue::sortQueue(){
    bool ok = false;
    while (!ok){
        ok = true;
        Node *current = getFirst();
        while (current->next != nullptr){
            if (current->item->toString() > current->next->item->toString()){
                BaseItem *temp = current->item;
                current->item = current->next->item;
                current->next->item = temp;
                ok = false;
            }
            current = current->next;
        }
    }
}

```

Рисунок 8 - Файл Queue.cpp

Вывод: Разработан полиморфный контейнер для безопасного хранения разнотипных данных. Использование механизма виртуальных функций и обработки исключительных ситуаций обеспечило отказоустойчивость программы. Корректность работы алгоритмов, включая сортировку, успешно проверена через графический интерфейс.

Часть 2. Преобразование контейнера в шаблон класса

Задание: Преобразовать разработанный контейнер в шаблон класса.

```
#ifndef MAINWINDOW_H
#define MAINWINDOW_H

#include <QMainWindow>
#include "queue.cpp"
#include <string>
#include <QMessageBox>
#include "queueexception.h"

QT_BEGIN_NAMESPACE
namespace Ui {
class MainWindow;
}
QT_END_NAMESPACE

class MainWindow : public QMainWindow
{
    Q_OBJECT

public:
    MainWindow(QWidget *parent = nullptr);
    void updateList();
    ~MainWindow();

private slots:
    void on_addBtn_clicked();

    void on_comboBox_currentIndexChanged(int index);

    void on_delBtn_clicked();

    void on_sortBtn_clicked();

private:
    Ui::MainWindow *ui;
    Queue<int> *numbers;
    SortedQueue *strings;
    int indexCB = 0;
};
#endif // MAINWINDOW_H
```

Рисунок 9 - Файл MainWindow.h


```

#ifndef QUEUEEXCEPTION_H
#define QUEUEEXCEPTION_H

#include <string>

class QueueException
{
    std::string message;
public:
    QueueException(std::string a): message(a) {}
    std::string getMessage();
};

class QueueEmptyException: public QueueException{
public:
    QueueEmptyException(): QueueException("Ошибка: попытка удаления из пустой очереди!"){}
};

class InvalidDataException: public QueueException
{
public:
    InvalidDataException():QueueException("Ошибка: введено некорректное число!"){}
};

#endif // QUEUEEXCEPTION_H

```

Рисунок 10 - Файл QueueException.h

```

#include "mainwindow.h"
#include "ui_mainwindow.h"

MainWindow::MainWindow(QWidget *parent)
    : QMainWindow(parent)
    , ui(new Ui::MainWindow)
{
    ui->setupUi(this);
    ui->sortBtn->setEnabled(false);
    numbers = new Queue<int>;
    strings = new SortedQueue;
}

MainWindow::~MainWindow()
{
    delete ui;
}

void MainWindow::on_addBtn_clicked()
{
    QString text = ui->lineEdit->text();
    bool ok;
    int value = text.toInt(&ok);
    if (ui->comboBox->currentIndex() == 0){
        try{
            if (ok) numbers->addItem(value);
            else throw InvalidDataException();
        }
        catch (InvalidDataException& e){
            QMessageBox::warning(this, "Warning", QString::fromStdString(e.getMessage()));
        }
    }
    else{
        strings->addItem(text.toStdString());
    }
    ui->lineEdit->clear();
    updateList();
}

void MainWindow::updateList(){
    ui->listWidget->clear();
    if (indexCB == 0){
        Node<int> *first = numbers->getFirst();
        Node<int> *current = first;
        while (current != nullptr){
            ui->listWidget->addItem(QString::number(current->item));
            current = current->next;
        }
    }
    else{
        Node<std::string> *first = strings->getFirst();
        Node<std::string> *current = first;
        while (current != nullptr){
            ui->listWidget->addItem(QString::fromStdString(current->item));
            current = current->next;
        }
    }
}

void MainWindow::on_comboBox_currentIndexChanged(int index)
{
    indexCB = index;
    if (indexCB) ui->sortBtn->setEnabled(true);
    else ui->sortBtn->setEnabled(false);
    updateList();
}

```

Рисунок 11 - Файл MainWindow.cpp

```

#include <string.h>
#include <stdio>
#include "baseitem.h"
#include "queueexception.h"

template <class type>
struct Node{
    type item;
    Node<type> *next;
    Node<type> *back;
};

template <class type>
class Queue{
    int size;
    Node<type> *first=nullptr;
    Node<type> *last=nullptr;

public:
    Queue(): size(0), first(nullptr) {}
    Queue(Node<type> *item): size(1), first(item) {}
    ~Queue(){
        Node<type> *current = first;
        while (current->next != nullptr) {
            Node<type> *temp = current;
            current = current->next;
            delete temp;
        }
    }

    void addItem(type it){
        if (first == nullptr)
        {
            first = new Node<type>{it, nullptr, nullptr};
            last = first;
            return;
        }
        Node<type> *current = first;
        while (current->next != nullptr) {
            current = current->next;
        }
        current->next = new Node<type>{it, nullptr, current};
        last = current->next;
        size += 1;
    }

    void delItem(){
        if (isEmpty()) throw QueueEmptyException();
        Node<type> *temp = first->next;
        delete first;
        first = temp;
    }

    bool isEmpty() const{
        if (first == nullptr) return 1;
        return 0;
    }
    void sortStrings();
    Node<type>* getFirst(){
        return first;
    }
};

```

Рисунок 12 – Реализация шаблона

```

class SortedQueue: public Queue<std::string>{
public:
    void sortQueue(){
        bool ok = false;
        while (!ok){
            ok = true;
            Node<std::string> *current = getFirst();
            while (current->next != nullptr){
                if (current->item > current->next->item){
                    std::string temp = current->item;
                    current->item = current->next->item;
                    current->next->item = temp;
                    ok = false;
                }
                current = current->next;
            }
        }
    };
};

```

Рисунок 13 – Сортированная очередь

Вывод: Выполнен успешный переход от полиморфизма к обобщенному программированию. Преобразование контейнера в шаблон класса позволило упростить архитектуру, обеспечить типобезопасность на этапе компиляции и повысить производительность кода без потери исходного функционала.